mmap

- 概述
  - 是一种内存映射，直接将文件里的内容映射到内存。 操作这块内存，就等价于操作这个文件
- 使用
  - mmap
    - #include <sys/mman.h>
      void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
    - void *addr: 内存从何处开始映射。建议直接写NULL
    - length/offset: 组合决定映射文件的哪块位置进来
    - int prot: 映射了之后，内存的权限。
      - PROT_READ: 读
      - PROT_WRITE | PROT_READ : 可读可写。 建议大家一起写上
    - int flags: 映射的内存的回写特点。
      - MAP_SHARED
        目前直接用它，保证修改完内存直接回写到文件中
    - int fd: 打开的文件描述符
    - 建议的参数
      - addr建议直接NULL
      - prot。如果只读PROT_READ； 读写 PROT_READ | PROT_WRITE
      - flags 建议直接写 MAP_SHARED
    - 返回值有点特殊
      - 如果成功，会返回指针。操作指针这块区域，就相当于操作文件
      - 如果失败，返回MAP_FAILED。
  - munmap
    - 传入获取到的void *
    - length和获取时的保持一致
- 一些坑
  - offset的取值问题 —— offset的取值，只能是4096的倍数。如果是其他值，这里会直接报错
  - 总线错误
    - 如果在mmap中，要访问的文件大小为0，但是却想映射几个字节，这时候就可能出现总线错误
    - 如果要映射的数目，大于了文件的实际大小。 这时候访问超出的部分，也可能出现
  - open的flag和mmap的prot
    - 如果只是想读数据， 这时候open用 O_RDONLY，mmap用PROT_READ
    - 如果想写数据，这时候open 用O_RDWR， mmap用 PROT_READ|PROT_WRITE
- fread&read&mmap对比
  - fread 有缓冲文件流。用来读文件的
  - read 无缓冲文件流。读文件的。
  - mmap 文件映射。
  - 如果说是操作普通文件， fread和read都可以。 read要注意一下读的数组大点。
    后续，在Linux下开发， read和write，可能出现的会更多。
    如果说要做大文件的复制。 这时候可以考虑使用mmap来提高效率。
  - 理论上， read比fread少一次数据拷贝； mmap比read少一次数据拷贝。
  - 数据复制了几次，要知道。
    有缓冲区文件流，拷了两次数据。 一次是从内核到用户态缓冲区，一次是用户态缓冲区到自己的数组。
    无缓冲区文件流，拷了一次数据。 从内核到自己的数组。
    mmap，没有拷贝数据。 我们拿到的指针，是内核开的后门。
- 案例
  - 小写转大写
    - 将文件中的小写字母，转成大写字母，其他的不动
    - 思路就是利用mmap将数据映射进来，挨个检查，是小写，就转成大写。 再放回原本的位置上
    - 在这个过程中，我们发现mmap的时候。 要传length。这时候学会另一个系统调用 fstat
      - 其他的参数一模一样，之前之前stat传path。现在传fd
      - 可以从buf里面，拿到st_size 就是大小
    - 如果使用read/write函数 —— 要注意一下，读取了之后，指针会往后挪。 所以有一个和fseek类似的函数， lseek
      大家晚上可以练习一下
  - 大文件的复制
    - 就意味着文件很大，我们无法一次弄进来。
    - 如果是使用read/write.其实无所谓，弄个循环使劲循环就完了。
      但是对于mmap，这种思路不行 —— 首先，先将文件大小获取到。 100M
      一次获取一点进来。 4M. len+offset
    - 思路
      - 既然不能一把整进来。这时候一点点弄进来。
        这时候要手动存储，已经读取的大小。以及偏移量各种信息，需要自己来处理
      - length和offset —— offset一定要是4K的倍数
      - 在这次复制中，copied_size，